Final Exam VT (Part 2)

Program Design and Data Structures (1DL201)

Teachers: Johannes Borgström and Eva Darulova

2023-08-23 / 14:00 — 18:00

Instructions

Read and follow these instructions carefully to increase your chance of getting good marks.

- This is a closed-book exam. You may use a standard English dictionary. Otherwise, **no notes, calculators, mobile phones, or other electronic devices are allowed**.
- Write your **exam code** at the top right corner of **each page**.
- There are **14 questions**. Each question can award 1 to 3 points. You can obtain a maximum of **36 points**.
- Some questions ask for a justification of your answer; your justification needs to relate the information given in the question, the course content, and your answer.
- Some questions ask you to write code; this code needs to conform to the coding convention. You only need to write function documentation if the question text states so.
- Write your answers directly on the question sheet. You may use English and Swedish.
- In case of questions, please ask the invigilator to call Eva or Johannes.

Question 1: Types (3 points)

What are the most specific types of the variables `x, y, z` in the following definitions? Do not use the `any` type, nor singleton types.

Answer:

```
const x: Array<number> = [1, 2, 3];
const y: Array<string | number> = [1, "foo", "bar"];
const z: (a: number) => string = a => a < 0 ? "negative" : "positive";
```

Question 2: Stacks (2 points)

Recall the stack data structure and its interface from the lectures:

```
export type NonEmptyStack<T> = [T, Stack<T>];
export type Stack<T> = null | [T, Stack<T>];
export function empty<T>(): Stack<T>
export function is_empty<T>(stck: Stack<T>): stck is null
export function push<T>(e: T, stck: Stack<T>): NonEmptyStack<T>
export function top<T>(stck: NonEmptyStack<T>): T
export function pop<T>(stck: NonEmptyStack<T>): Stack<T>
```

and assume the following statements in a file:

```
import { empty, is_empty, pop, push, top } from '../lib/stack';
```

For the following expressions, state the result of the expression, or explain why the expression results in a type error:

Answer:

```
top(push('c', pop(push(2, push('a', empty<string>()))))); 
```

type error: it is not possible to push a number on top of a stack of type Stack<string>

```
const s0 = pop(push('c', push('b', push('a', empty<string>()))));
is_empty(s0) ? null : top(s0);
```

b

Question 3: Queues (2 points)

Recall the queue data structure and the interface of its mutable implementation:

```
function empty<T>(): Queue<T>
function is_empty<T>(q: Queue<T>): boolean
function enqueue<T>(e: T, q: Queue<T>)
function head<T>(q: Queue<T>): T
function dequeue<T>(q: Queue<T>)
```

Complete the following Jest testcases for unit testing the queue implementation. Make sure that your testing code is correctly typed.

Answer:

```
import { empty, enqueue, dequeue, head, is_empty } from '../lib/queue_array';

test('queue operations', () => {
    const que = empty<number>();
    enqueue(17, que);
    dequeue(que);
    enqueue(25, que);
```

```
    enqueue(3, que);
    dequeue(que);
    expect(head(que)).toBe(3);
});

test('empty queue test', () => {
    const que = empty<number>();
    enqueue(5, que);
    enqueue(5, que);
    dequeue(que);
    expect(is_empty(que)).toBe(false);
});
```

Question 4: Preconditions (2 points)

Consider the following function:

```
/**
 * Checks whether a list is sorted in ascending order. Duplicate elements
 * following each other are considered sorted.
 * @precondition ???
 * @example
 * is_sorted(list(1, 2, 3, 4, 4, 5)) // returns true
 * is_sorted(list(1, 5, 3, 4, 4, 5)) // returns false
 * @param lst
 * @returns true if lst is sorted, false otherwise
 */
function is_sorted(lst: List<number>): boolean {
    function helper(l: List<number>, previous: number): boolean {
        return is_null(l)
               ? true
               : head(l) >= previous
               ? helper(tail(l), head(l))
               : false;
    }
    return is_null(lst) ? true : helper(lst, head(lst));
}
```

What is a suitable precondition for `is_sorted`? Write `true` if you think the precondition can be removed.

Answer:

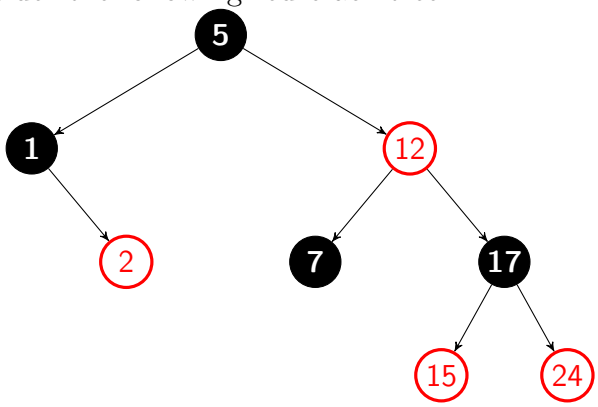
true

Justify your answer:

The function works as expected for all possible inputs that are allowed by the types, including corner cases such as the empty list.

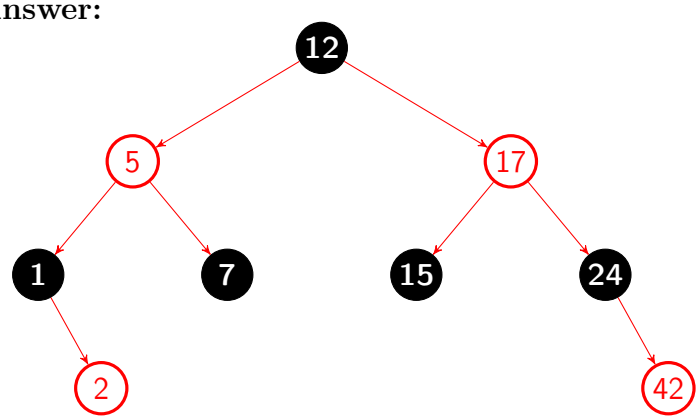
Question 5: Red-black trees (3 points)

Consider the following red-black tree:



What is the resulting tree after inserting an element with key 42? Show the intermediate steps. Circle red nodes, and leave black nodes without a border (do not use a red pen).

Answer:



Question 6: Complexity (3 points)

What is the **worst-case time** complexity in terms of Big-Theta of the following operations?
Answer:

- pushing an element on top of a stack with n elements: $\Theta(1)$
- retrieval of the element with highest priority from a heap with n elements: $\Theta(log\ n)$
- checking whether an edge exists in a graph with v nodes and e edges that is represented with *adjacency lists*: $\Theta(v)$

Question 7: Records (2 points)

Recall the generic type `List<T>`. Use it to define a record type for directed graphs, using the adjacency list representation. Use informative names for the fields.
Answer:

```
type DirectedListGraph = {
  nodes : number,
  edges : Array<List<number>>
} ;
```

Question 8: Heaps (3 points)

Remove the top element from the following array heap using the algorithm seen in class (called MAX-HEAP-EXTRACT-MAX in I2A), showing the contents of the array at each intermediate step.
Answer:

12	9	11	7	3	10	8	(input array)
8	9	11	7	3	10		
11	9	8	7	3	10		
11	9	10	7	3	8		(array after last step)

Question 9: Hashtables (3 points)

Assume that you have a hash table with 5 slots that uses the hash function $h(s) = (\texttt{length}\ s)^2$ and open addressing with linear probing to resolve collisions. Let $\perp$ denote an empty cell, and Δ a deleted one.
Perform the following three operations, and show the resulting table after each of them.
`insert("lighter")`, **`delete`**`("thunder")`, `insert("veryer")`.

Answer:

0	1	2	3	4	
fandangos	Δ	$\perp$	$\perp$	thunder	(initial table)

0	1	2	3	4	
fandangos	lighter	$\perp$	$\perp$	thunder	(after inserting "lighter")

0	1	2	3	4
fandangos	lighter	$\perp$	$\perp$	Δ

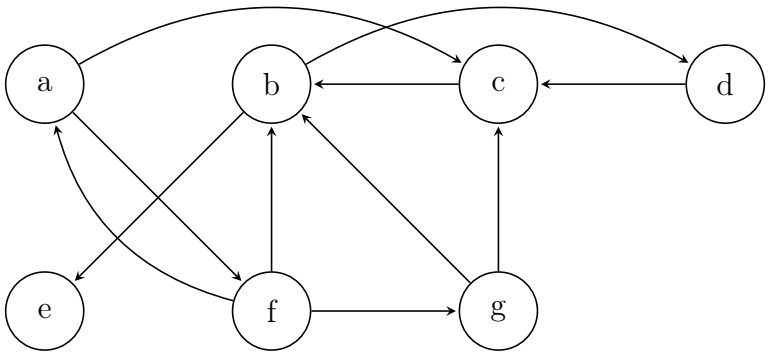
(after deleting "thunder")

0	1	2	3	4
fandangos	lighter	veryer	$\perp$	Δ

(final table)

Question 10: Graph representation (2 points)

Consider the following graph:



Represent this graph as an adjacency matrix.

Answer:

	a	b	c	d	e	f	g
a			1			1	
b				1	1		
c		1					
d			1				
e							
f	1	1					1
g		1	1				

Question 11: Graph traversal (2 points)

Consider the graph in question 10. Perform a depth-first search in the graph, using the restart order d, e, f, g, a, b, c. What are its discovery and finish orders (i.e., in which order are the nodes coloured grey and black, respectively)?

Answer:

d	c	b	e	f	g	a

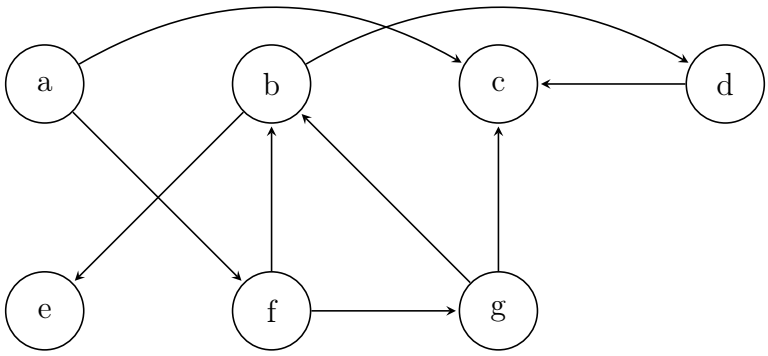
(discovery order)

e	b	c	d	g	a	f

(finish order)

Question 12: Topological sort (3 points)

Consider the following graph:



Can node c be before node b in a topological sort of this graph?
Motivate your answer, beginning from the definition of a topological sort.

Answer:

No. In a topological sort, if there is an edge from node X to node Y then X appears before Y. In the graph there are edges from b to d and from d to c, so b must come before d and d must come before c, so b is always before c.

Question 13: Strongly connected component (3 points)

Let the nodes of a graph G be the integers $1, \ldots, 10$, and let there be an edge in G from node a to node b iff a divides b^2 (i.e., if the remainder when dividing b^2 with a is 0).
What are the strongly connected components of G?
Motivate your answer by giving a cycle that spans each of the SCCs.
1, $2 \to 4 \to 8 \to 4 \to 2$, $3 \to 9 \to 3$, 5, 6, 7, 10.

Question 14: Meta-circular Evaluator (3 points)

Recall the high-level sketch of the metacircular evaluator from the lecture. Complete the sketch below using the following expressions:

1. `const result = evaluate(program, env);`
2. `display("Nothing to do!"");`
3. `const program_ir = parse(program);`
4. `return driver_loop(env);`
5. `const program = read_input_from_user();`
6. `display(result);`

You may use the numbers of the statements, instead of writing out the statements in full.
Answer:

```
function driver_loop(env) {
    const program = read_input_from_user();        // 5
    if (program !== "") {
        const program_ir = parse(program);         // 3
        const result = evaluate(program_ir, env);  // 1
        display(result);                           // 6
        return driver_loop(env);                   // 4
    } else {
        display("Nothing to do!");                 // 2
   }
}
```

This is the final page.